



Tema 1: Programación multiproceso

INDICE

1. Proceso

2. Hilo

3. Concurrencia

4. Ejemplos

5. Métodos clase Thread

1. Proceso

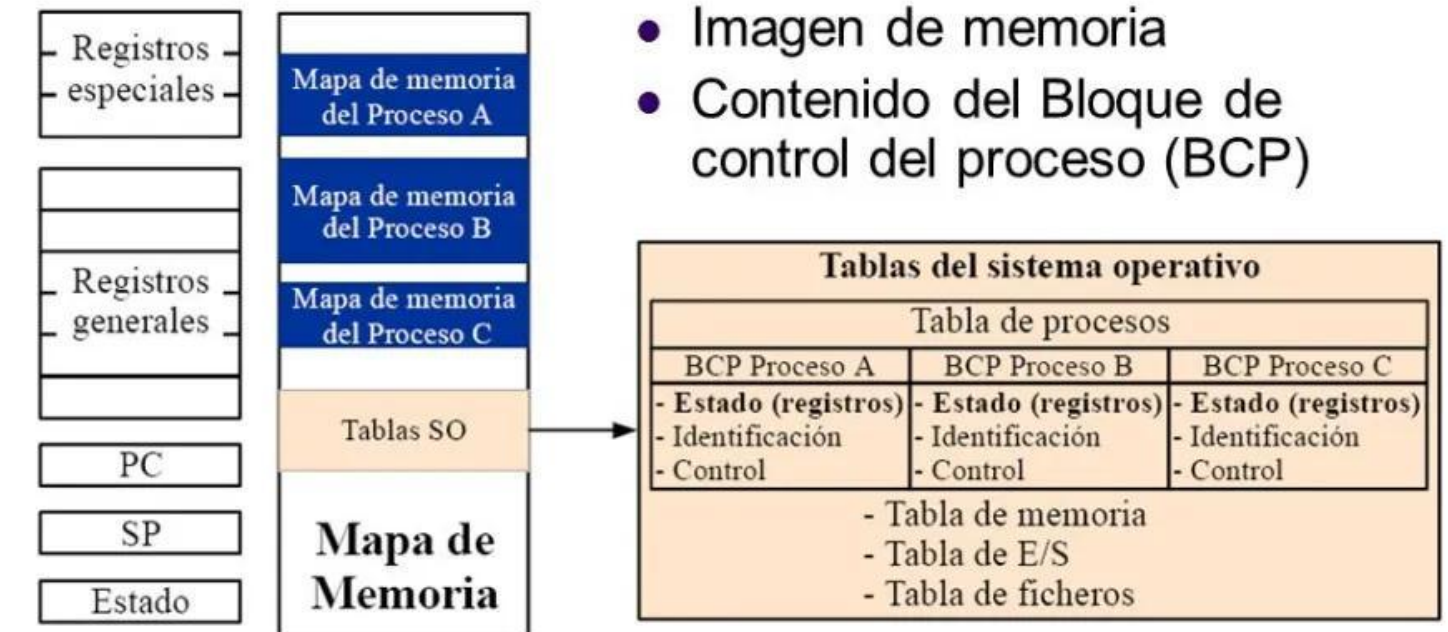
- Programa en ejecución gestionado por el Sistema Operativo.
- Un proceso = **programa ejecutándose + recursos asignados**.

◆ Características

- Tiene su propio **espacio de memoria**.
- Requiere **recursos**: CPU, memoria, archivos, dispositivos.
- Posee un **estado** (nuevo, listo, en ejecución, bloqueado, terminado).

Información del proceso

- Estado del procesador
- Imagen de memoria
- Contenido del Bloque de control del proceso (BCP)



1. Hilo

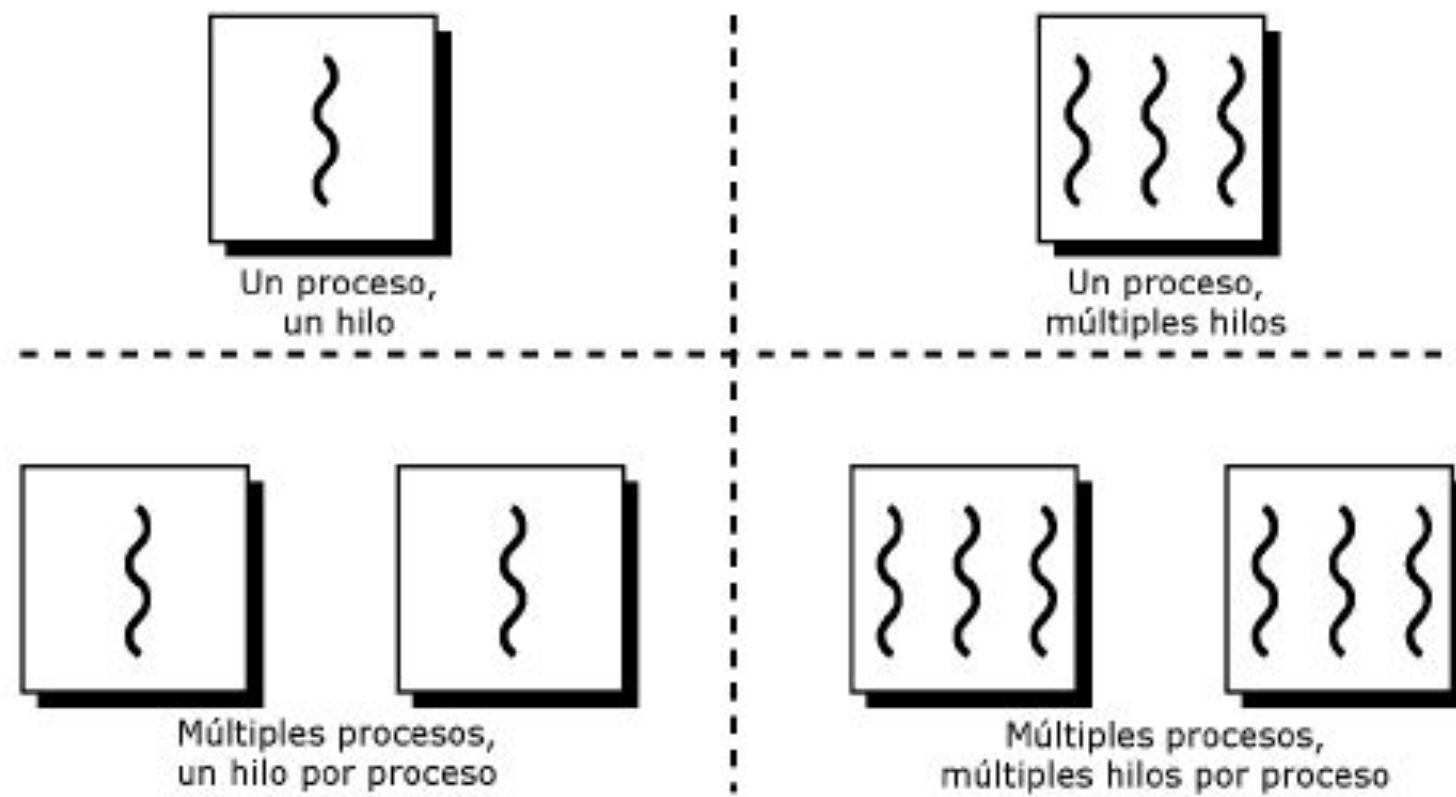
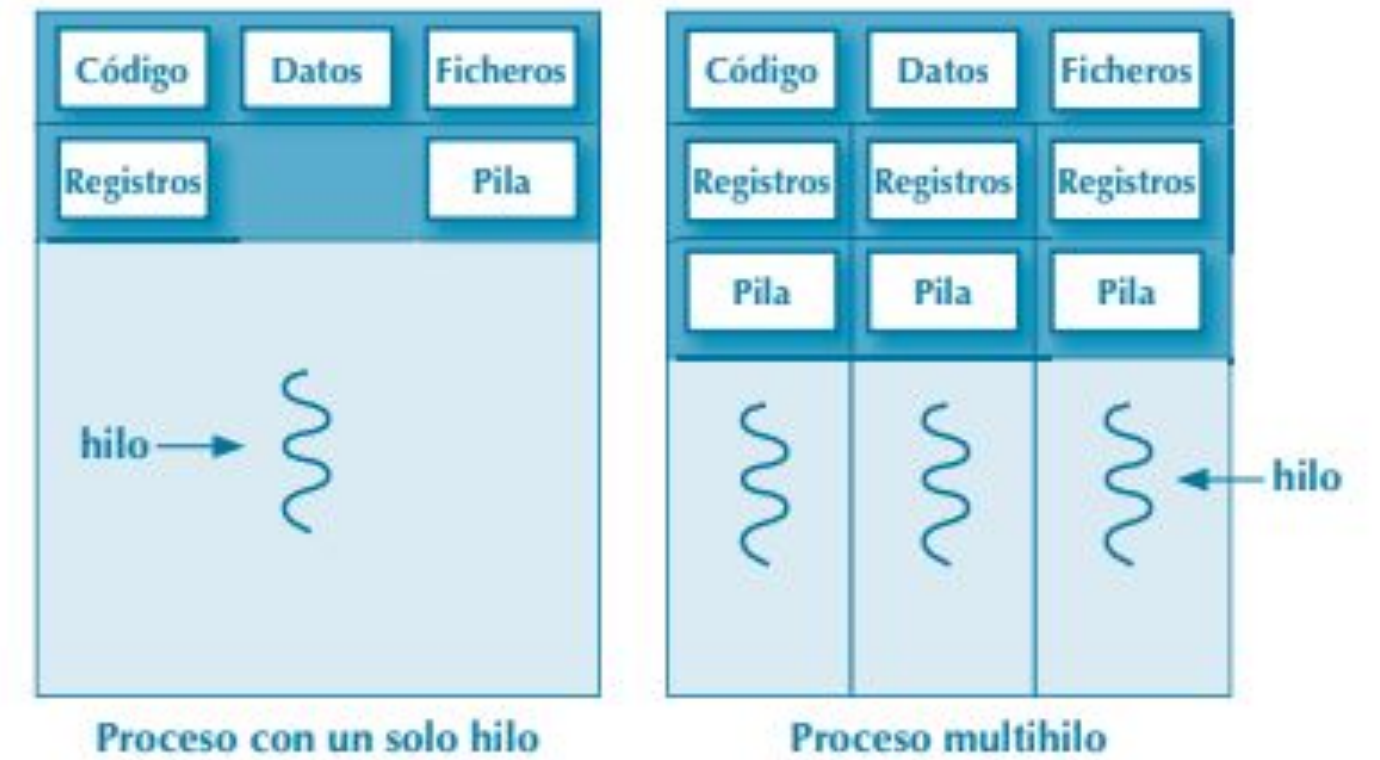


Figura 2.1
Procesos con un solo hilo y multihilo.

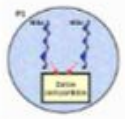


1. Hilo

DIFERENCIAS ENTRE HILO Y PROCESOS

Procesos e
Hilos

¿Cuál es la

PROCESOS	HILOS
- Es un programa en ejecución ✓ - Están completamente aislados y no comparten memoria ✓ - Consumen más recursos que los hilos ✓ - Son menos eficientes en el contexto de la comunicación ✓ - Requiere más tiempo para ser creado ✓ - En caso de error en el computador resulta en la pérdida completa del proceso ✓	 HILOS - Es un proceso ligero o parte de uno ✓ - Si comparten la memoria entre sí ✓ - Consumen menos recursos que los procesos convencionales ✓ - Mejoran la eficiencia en el contexto de la comunicación ✓ - Requiere menos tiempo en ser creado ✓ - En caso de error en el computador un hilo puede ser recuperado a diferencia de un proceso convencional ✓

Proceso 1

Stack

Hilo 1

Stack
Registers
Code

Hilo 2

Stack
Registers
Code

Hilo 3

Stack
Registers
Code

1. Concurrency – Parallelism – Distribution

◆ Concurrency

- Varios procesos/hilos *avanzan en el tiempo*, pero no necesariamente a la vez.
- El sistema operativo intercala su ejecución.
- Ejemplo: en un PC de un solo núcleo, alternar entre abrir un navegador y escuchar música.

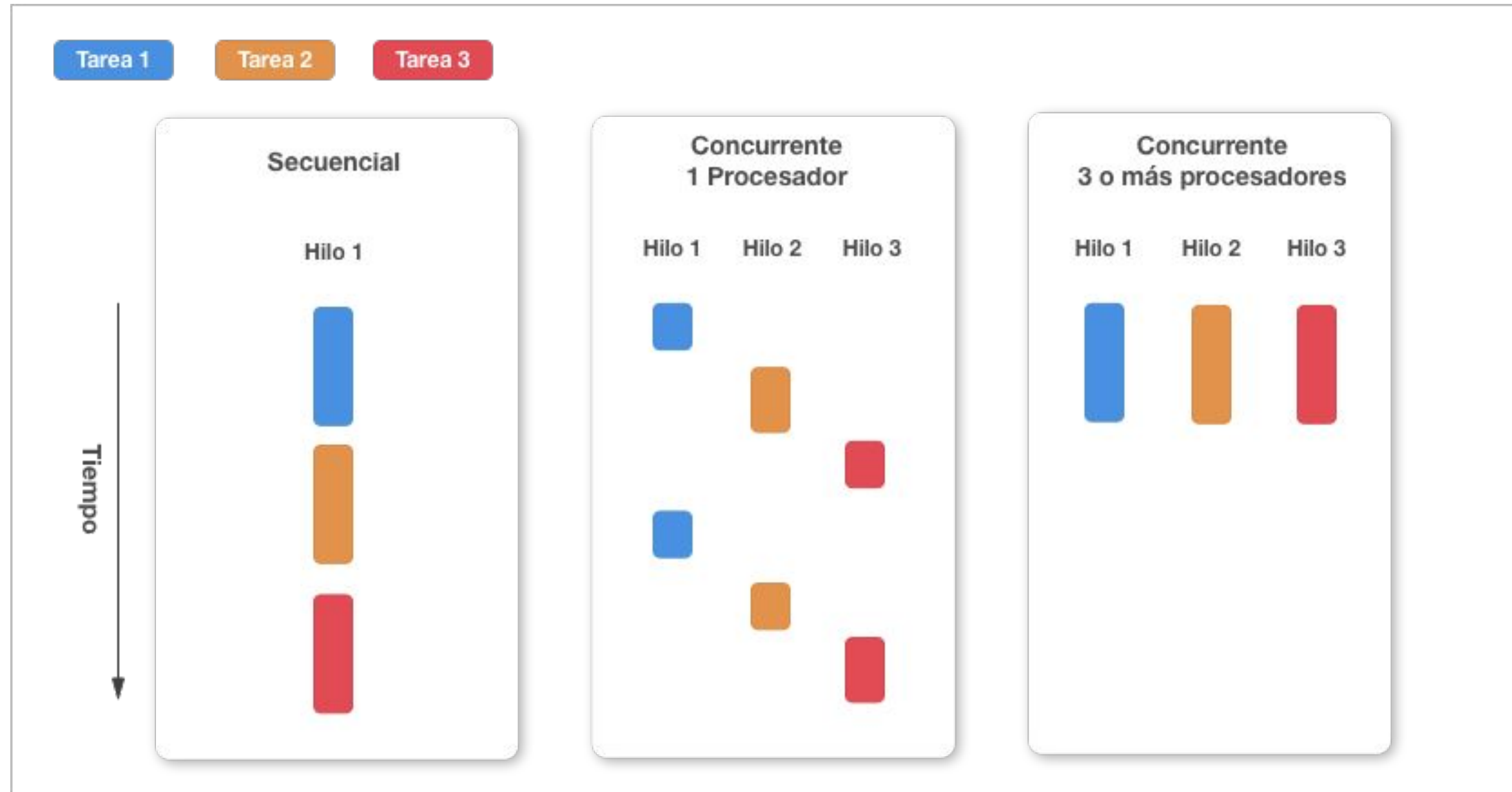
◆ Parallelism

- Varios procesos/hilos se ejecutan *realmente al mismo tiempo*.
- Requiere procesadores o núcleos múltiples.
- Ejemplo: cálculos científicos en varios cores.

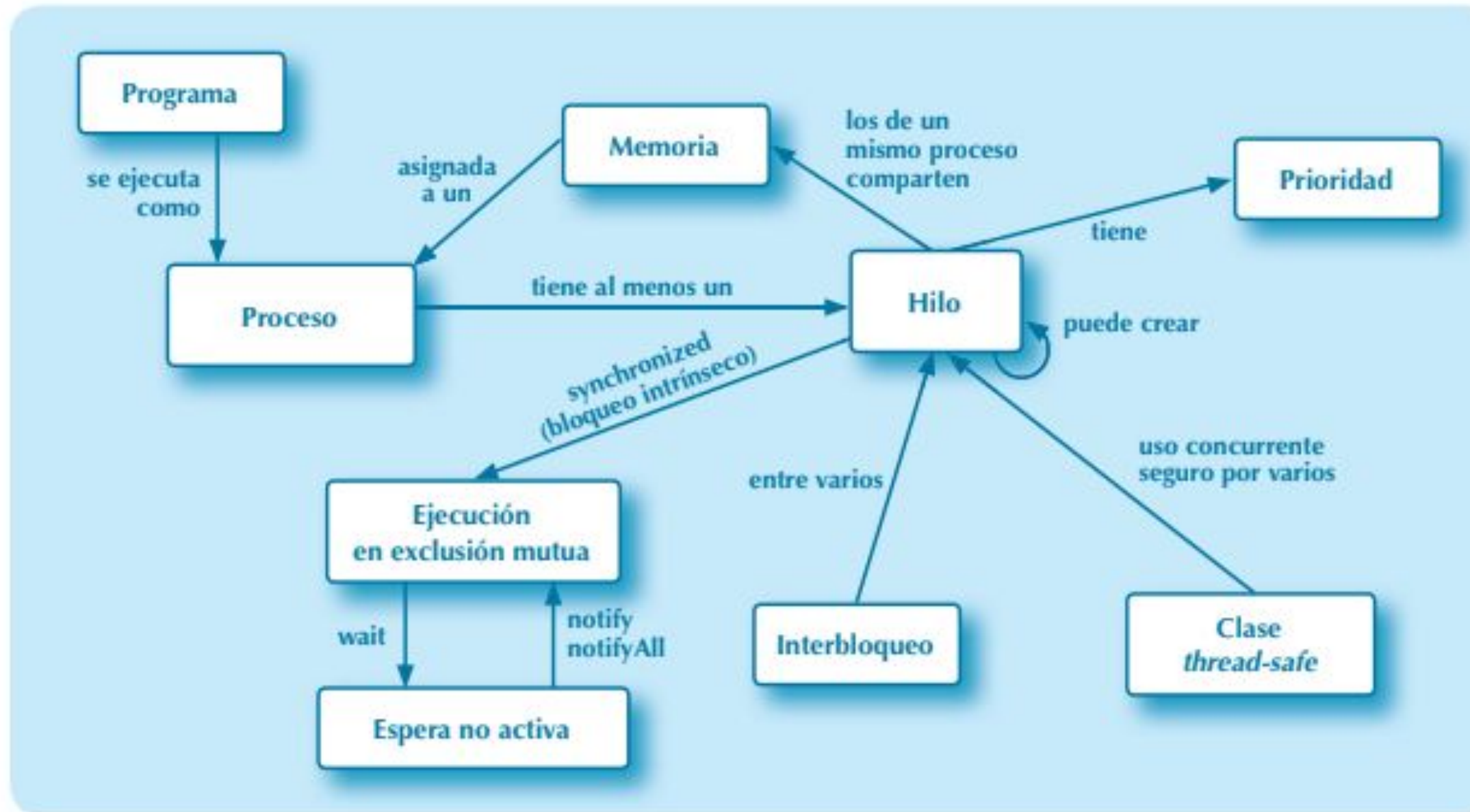
◆ Distribution

- Ejecución de tareas en *varios equipos conectados por red*.
- Requiere coordinación y comunicación entre nodos.
- Ejemplo: Google, Netflix, videojuegos online.

1. Concurrency – Parallelismo – Distribución



1. Concurrency – Parallelismo – Distribución



1. Ejemplo práctico - Proceso

```
import java.io.*;

public class EjecutarProceso {
    public static void main(String[] args) {
        try {
            // Crear un proceso que ejecute el comando "notepad" (en Windows)
            // En Linux/Mac podrías usar "gedit" o "ls"
            ProcessBuilder pb = new ProcessBuilder("notepad");
            Process proceso = pb.start();

            // Esperar a que el proceso termine
            int exitCode = proceso.waitFor();
            System.out.println("El proceso terminó con código: " + exitCode);

        } catch (Exception e) {
            e.printStackTrace();
        }
    }
}
```

1. Ejemplo práctico - Hilo

```
class Tarea extends Thread {  
    public void run() {  
        System.out.println("Ejecutando hilo: " + getName());  
    }  
}  
  
public class Main {  
    public static void main(String[] args) {  
        new Tarea().start(); // Se lanza un hilo  
    }  
}
```

1. Métodos clase Thread

MÉTODOS DE LA CLASE `Thread`

Método	Funcionalidad
<code>void run()</code>	Se ejecuta cuando se lanza el hilo. Es el punto de entrada del hilo, como el método <code>main()</code> es el punto de entrada del proceso.
<code>void start()</code>	Lanza el hilo. La JVM crea el hilo y ejecuta su método <code>run()</code> .
<code>static void sleep(long ms)</code> <code>static void sleep(long ms, long ns)</code>	Detiene la ejecución del hilo actualmente en ejecución durante un tiempo, que se puede indicar en microsegundos o en una combinación de microsegundos y nanosegundos.
<code>void join()</code> <code>void join(long ms)</code> <code>void join(long ms, long ns)</code>	Espera a que termine el hilo. Se puede indicar un tiempo máximo de espera, bien en milisegundos, bien en una combinación de milisegundos y nanosegundos.

1. Métodos clase Thread

<code>public void interrupt()</code> <code>public boolean isInterrupted()</code> <code>public static boolean interrupted()</code>	El primer método interrumpe la ejecución de un hilo. El segundo método verifica si se ha interrumpido un hilo. El tercer método (estático) verifica si se ha interrumpido la ejecución del hilo actual, y borra el estado de interrupción, de manera que una llamada posterior devolvería <code>false</code> , a menos que se vuelva a interrumpir.
<code>boolean isAlive()</code>	Comprueba si el hilo está vivo. Un hilo está vivo cuando se ha iniciado y no ha terminado su ejecución.
<code>int getPriority()</code> <code>void setPriority(int nuevaPrior)</code>	Se puede asignar una prioridad a un hilo, y se puede obtener la prioridad de un hilo.
<code>static Thread currentThread()</code>	Devuelve un objeto de clase <code>Thread</code> correspondiente al hilo en ejecución actualmente.
<code>long getId()</code>	Devuelve el identificador del hilo.
<code>String getName()</code> <code>void setName(String nombre)</code>	Se puede asignar un nombre a un hilo, y se puede recuperar el nombre del hilo.
<code>Thread.State getState()</code>	Devuelve el estado del hilo.
<code>boolean isDaemon()</code> <code>void setDaemon(boolean on)</code>	Un hilo puede ser de tipo <i>daemon</i> . La distinción es importante, porque la JVM termina su ejecución cuando no queda ningún hilo activo o cuando solo quedan hilos de tipo <i>daemon</i> .